فصل ۸: بردار پشتیبان

مقدمه

در این فصل، با استفاده از یک ماشین بردار پشتیبان (SVM) با یک هسته خطی شروع می‌کنیم تا نحوه کار آن‌ها را نشان دهیم. SVMها یک ابر صفحه یا یک سطح خطی در ابعاد مختلف ایجاد می‌کنند که به بهترین وجه داده‌ها را جدا می‌کند. در فضای دو بعدی، ابر صفحه خطی است که داده‌ها را جدا می‌کند. در ادامه فصل، SVMهایی با هسته (RBF) استفاده می‌شوند. با وجود اینکه هسته‌ها غیرخطی هستند، اما دارای سطوح جدا کننده صاف می‌باشند. در عمل، SVMها با بسیاری از مجموعه داده‌ها به خوبی کار می‌کنند.

طبقه‌بندی داده‌ها با SVM خطی

در حالی که SVMها تفسیر احتمالی ساده‌ای ندارند، اما تفسیر بصری- هندسی ساده‌ای دارند. ایده اصلی SVMهای خطی این است که دو کلاس را با بهترین صفحه ممکن جدا کند. حال برای روشن شدن مطلب بیایید با SVM کلاسه‌بندی دو کلاسه انجام دهیم.

بیایید با بارگیری و تجسم مجموعه داده iris موجود شروع کنیم

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt #Library for visualization
from sklearn import datasets
iris = datasets.load_iris()
```

```
X_w = iris.data[:, :2] #load the first two features of the iris data
y_w = iris.target #load the target of the iris data
```

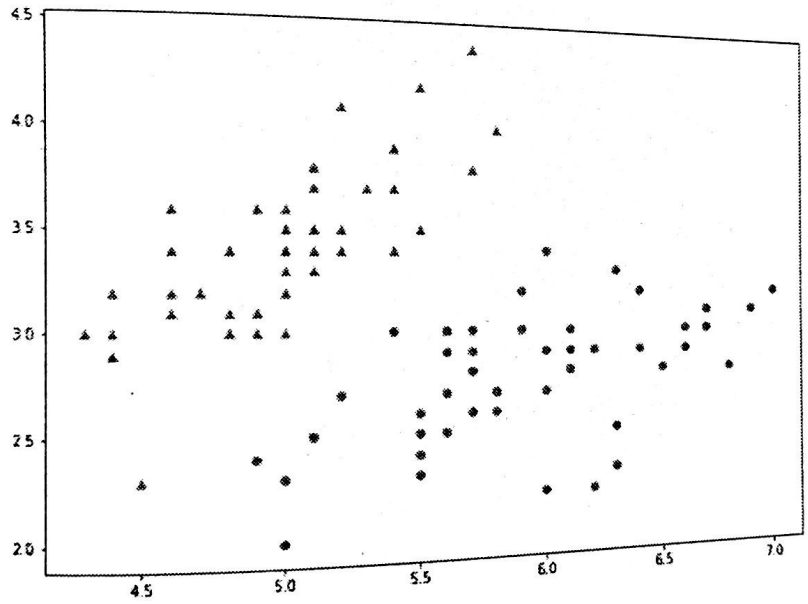
فقط دو کلاس اول را برای مجموعه ویژگی و مجموعه هدف انتخاب کنید

```
X = X_w[y_w < 2]
y = y_w[y_w < 2]
```

X_1 با کلاس ۱ مطابقت دارد، و در ادامه نمودار داده‌ها را رسم می‌کنیم.

در کد زیر، X_0 به زیر مجموعه‌ای از ورودی‌ها اطلاق می‌شود که با کلاس صفر و

```
X_0 = X[y == 0]
X_1 = X[y == 1]
%matplotlib inline
plt.figure(figsize=(10,7)) #change figure-size for easier viewing
plt.scatter(X_0[:,0],X_0[:,1], color = 'red' , marker= '^')
plt.scatter(X_1[:,0],X_1[:,1], color = 'blue')
```



شکل (۸-۱): نمایش داده‌های دو کلاس

روند یافتن خط SVM ساده است. این فرآیند همانند هر ارزیابی‌کننده دیگر است:

۱. مجموعه‌های آموزشی و آزمایشی ایجاد کنید.
۲. یک نمونه مدل SVM ایجاد کنید.
۳. مدل SVM را با داده‌ها برازش کنید.
۴. با مدل SVM، داده‌های جدید را پیش‌بینی کنید و عملکرد مدل را اندازه‌گیری کنید.

مجموعه داده را محدود به دو ویژگی اول از دو کلاس ابتدایی می‌کنیم.

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=0.2,
random_state=7,stratify=y)
```

یک نمونه مدل SVM با هسته خطی ایجاد می‌کنیم.

```
from sklearn.svm import SVC
svm_inst = SVC(kernel='linear')
svm_inst.fit(X_train,y_train)
```

پیش‌بینی را انجام می‌دهیم:

```
y_pred = svm_inst.predict(X_test)
```

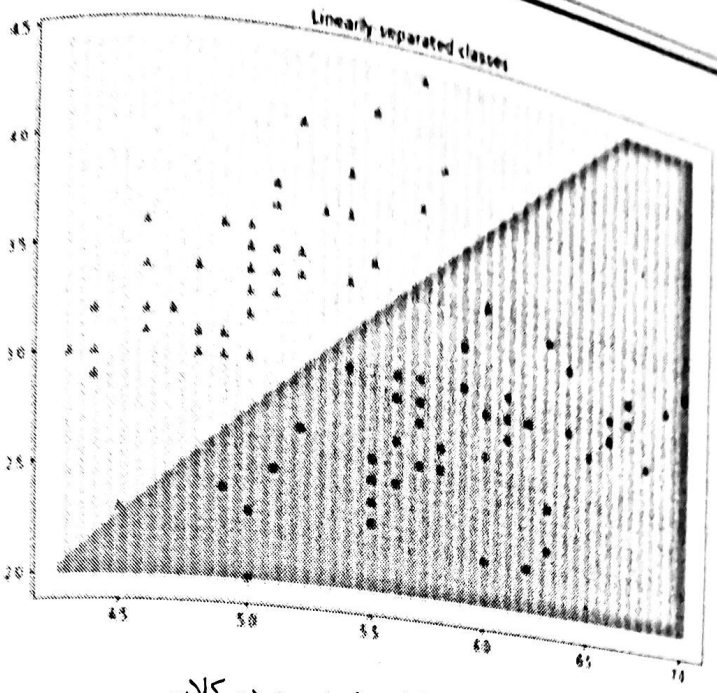
عملکرد SVM را روی مجموعه آزمایش اندازه‌گیری می‌کنیم:

```
from sklearn.metrics import accuracy_score
accuracy_score(y_test, y_pred)
output:1
```

به طور عالی توانست داده‌های تست را دسته‌بندی کرد. این تعجب‌آور نیست، زیرا وقتی هر کلاس را تصویر می‌کنیم، جدا کردن آن‌ها از نظر بصری آسان بود.

حال با استفاده از تخمین زننده در یک شبکه دو بعدی، مرز تصمیم‌گیری، خط جدا کننده کلاس‌ها را رسم می‌کنیم.

```
from itertools import product
#Minima and maxima of both features
xmin, xmax = np.percentile(X[:, 0], [0, 100])
ymin, ymax = np.percentile(X[:, 1], [0, 100])
#Grid/Cartesian product with itertools.product
test_points = np.array([[xx, yy] for xx, yy in
product(np.linspace(xmin, xmax), np.linspace(ymin, ymax))])
#Predictions on the grid
test_preds = svm_inst.predict(test_points)
X_0 = X[y == 0]
X_1 = X[y == 1]
%matplotlib inline
plt.figure(figsize=(10,7)) #change figure-size for easier viewing
plt.scatter(X_0[:,0],X_0[:,1], color = 'red', marker='^')
plt.scatter(X_1[:,0],X_1[:,1], color = 'blue')
colors = np.array(['r', 'b'])
plt.scatter(test_points[:, 0], test_points[:, 1],
color=colors[test_preds], alpha=0.25)
plt.scatter(X[:, 0], X[:, 1], color=colors[y])
plt.title("Linearly-separated classes")
```



شکل (۸-۲): نمایش ابرصفحه بین دو کلاس

در بعضی مواقع، پیش‌بینی در یک مجموعه کامل از نقاط ممکن است از نظر محاسباتی گران باشد، به خصوص اگر SVM کلاس‌های زیادی را در ابعاد مختلف پیش‌بینی کند. در این موارد، شما نیاز به دسترسی به اطلاعات هندسی مرز تصمیم SVM خواهید داشت.

یک مرز تصمیم‌گیری خطی و یک ابر صفحه، توسط یک بردار نرمال نسبت به ابر صفحه و یک عرض از مبدا مشخص می‌شود. بردارهای نرمال در ویژگی coef_data نمونه SVM موجود است. عرض از مبداها در ویژگی intercept_ data نمونه SVM موجود می‌باشد. با دستور زیر این دو ویژگی را مشاهده می‌کنیم:

```
print("coef" ,svm_inst.coef_)
print("intercept" ,svm_inst.intercept_)
output: coef [[ 2.22246001 -2.2213921 ]]
intercept [-5.00384439]
```

شاید بتوانید به سرعت ببینید که بردار coef_[0] عمود بر خطی است که رسم کردیم تا هر دو کلاس را از هم جدا کند. svm_inst.intercept_ و svm_inst.coef_ دارای تعداد ردیف یکسان می‌باشند. هر سطر مربوط به یک صفحه می‌شود که کلاس‌ها را جدا می‌کند. در مثال، دو کلاس به صورت خطی با یک ابر صفحه از هم جدا شده است. نوع خاص SVM، SVC است که یک طبقه‌بندی یک _در مقابل _یک (one-versus-one) را اجرا می‌کند: یک صفحه منحصر به فرد ترسیم می‌کند که هر جفت کلاس را تفکیک می‌کند. اگر می‌خواستیم سه کلاس را جدا کنیم، سه ترکیب ممکن وجود دارد، 3= 2/ 2× 3. برای کلاس‌ها، تعداد صفحه‌های ارائه شده توسط SVC به شرح زیر است

$$\text{صفحات تعداد} = \frac{Number\ of\ classes \times (Number\ of\ classes - 1)}{2}$$

تعداد ستونها در ویژگی coef_ data تعداد ویژگی‌های موجود در داده است که در این حالت دو است.

برای یافتن تصمیم در مورد یک نقطه از فضا، معادله زیر را برای صفر حل کنید

جدول (۸-۱): تعیین جهت بردار

$C\vec{v} + \vec{i}$ $\vec{v}$ مربوط به مرز تصمیم است	C همان coef_array که یک بردار در فضا است و i عرض از مبدا می‌باشد
بردار v در یک سمت صفحه قرارداد	$C\vec{v} + \vec{i} > 0$
بردار v روی صفحه قرارداد	$C\vec{v} + \vec{i} = 0$
بردار v در سمت دیگر صفحه قرارداد	$C\vec{v} + \vec{i} < 0$

علاوه بر این، می‌توانید پارامترهای نمونه را مشاهده کنید تا درباره آن اطلاعات بیشتری کسب کنید

```
svm_inst
SVC(kernel='linear')
```

به طور سنتی، عملکرد پیش‌بینی SVC از طریق پارامترهای زیر بهینه می‌شود:

مقدار C، گاما، و شکل هسته. C حاشیه SVM را توصیف می‌کند و به طور پیش فرض روی یک تنظیم می‌شود. حاشیه، فضای خالی در دو طرف ابر صفحه است و هیچ داده‌ای در بین آن قرار ندارد. اگر مجموعه داده‌های شما نویز زیادی داشته باشند، مقادیر C بزرگ‌تر با اعتبار متقابل را امتحان کنید. C متناسب با خطای موجود در حاشیه است و هرچه مقدار C بالاتر می‌رود، SVM سعی می‌کند حاشیه را کوچک‌تر کند.

نکته آخر در مورد SVM این است که می‌توانیم داده‌ها را مقیاس‌بندی مجدد کنیم و مقیاس‌گذاری را با اعتبارسنجی متقابل آزمایش کنیم. به راحتی، مجموعه داده iris برای همه ورودی‌ها دارای واحد cms است بنابراین مقیاس‌بندی مجدد لازم نیست اما برای یک مجموعه داده دلخواه باید آن را بررسی کنید.

برای این مثال، ما با مجموعه داده iris ادامه خواهیم داد، اما از دو کلاس که تشخیص آن‌ها دشوارتر است، استفاده خواهیم کرد، گونه‌های Versicolour و Virginica.

در این بخش ما به موارد زیر توجه خواهیم کرد:

- راه‌اندازی خط لوله: زنجیره‌ای از تحولات با یک مدل پیش‌بینی

• جستجوی شبکه: اسکن عملکرد چندین نسخه SVM با پارامترهای مختلف

داده‌ها را بارگذاری کرده و آن‌ها را به دو مجموعه آزمایش و تست تقسیم می‌کنیم

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
iris = datasets.load_iris()
X_w = iris.data[:, :2] #load the first two features of the iris data
y_w = iris.target #load the target of the iris data
X = X_w[y_w != 0]
y = y_w[y_w != 0]
X_1 = X[y == 1]
X_2 = X[y == 2]
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,test_size=0.2,
random_state=7,stratify=y)
```

سپس یک خط لوله با دو مرحله احداث کنید: یک مرحله مقیاس‌گذاری و یک مرحله SVM. بهتر است داده‌ها قبل از انتقال به SVM مقیاس‌بندی شوند.

```
from sklearn.svm import SVC
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
svm_est = Pipeline([('scaler',StandardScaler()),('svc',SVC())])
```

نکته:

توجه داشته باشید که در "خط لوله"، مرحله "مقیاس‌گذاری" دارای نام مقیاس ساز و SVM دارای نام svc است. نام‌ها در مرحله بعدی بسیار مهم خواهند بود. توجه داشته باشید که SVM پیش فرض یک RBF است که غیرخطی است.

حال پارامترهای گاما و c را تعریف می‌کنیم

```
Cs = [0.001, 0.01, 0.1, 1, 10]
gammas = [0.001, 0.01, 0.1, 1, 10]
```

در آخر، برای شبکه پارامتر فرهنگ لغت استفاده می‌کنیم. نام کلیدهای پارامتر SVM با svc__ شروع می‌شوند، پس از آن نام پارامتر در تخمین زننده، مانند C و gamma دنبال می‌شود

```
param_grid = dict(svc__gamma=gammas, svc__C=Cs)
```

پارامتر n_splits به تعداد تقسیم‌ها اشاره می‌کند، یا تلاش می‌کند، مجموعه داده به آن تعداد تقسیم شود. پارامتر test_size مقدار داده‌ای است که برای آزمایش باقی مانده است. با استفاده از مجموعه آزمون در هر بخش، تخمین زننده امتیاز دهی خواهد شد

```
from sklearn.model_selection import StratifiedShuffleSplit
cv = StratifiedShuffleSplit(n_splits=5, test_size=0.2,random_state=7)
```

برای تعیین تعداد بخش ها(fold) cv را برابر ۱۰ قرار می‌دهیم

```
cv = 10
```

برای جستجوی شبکه سه عنصر مورد نیاز است:

- تخمین زننده
- یک شبکه پارامتر
- طرح اعتبارسنجی متقابل

ما این سه عنصر را داریم. جستجوی شبکه را تنظیم کنید. آن را روی مجموعه آموزش اجرا کنید

```
from sklearn.model_selection import GridSearchCV
grid_cv = GridSearchCV(svm_est, param_grid=param_grid, cv=cv)
grid_cv.fit(X_train, y_train)
grid_cv.best_params_
output: {'svc__C': 10, 'svc__gamma': 0.1}
grid_cv.best_score_
0.7125
```

جایگزین شبکه جستجوی تصادفی

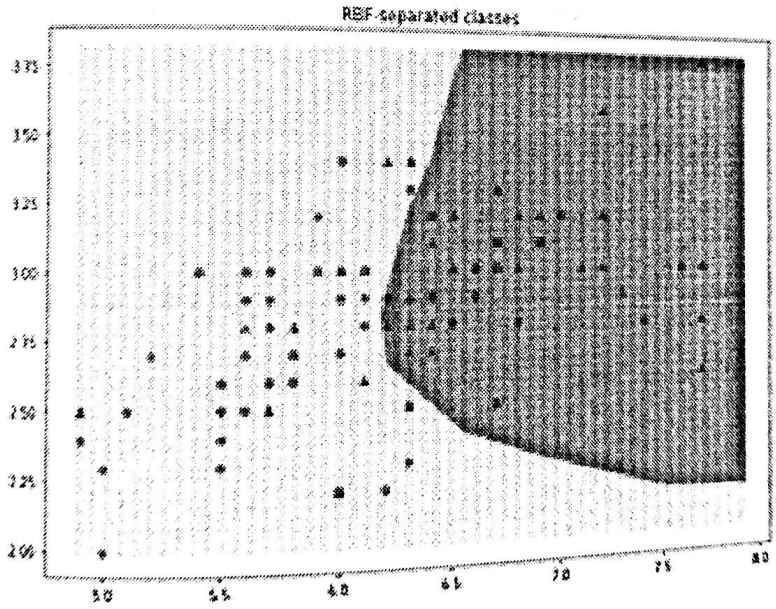
GridSearchCV اسکن کاملی را برای بهترین مجموعه پارامترها برای تخمین زننده انجام می‌دهد. در این حالت، 25=5×5 جفت‌های (C، گاما) مشخص شده توسط param_grid را جستجو می‌کند. استفاده از RandomizedSearchCVیک گزینه جایگزین GridSearchCV بود.

```
from sklearn.model_selection import RandomizedSearchCV
rand_grid = RandomizedSearchCV(svm_est,
param_distributions=param_grid,cv=cv,n_iter=10)
#Ahmad Azarnik
rand_grid.fit(X_train, y_train)
rand_grid.best_params_ # مشاهده c, gamma
output: {'svc__gamma': 0.1, 'svc__C': 10}
```

نمودار مرز تصمیم RBF غیرخطی

مرز تصمیم RBF را با کدی مشابه دستورالعمل قبلی رسم کنید. ابتدا یک شبکه ایجاد کرده و پیش‌بینی کنید که هر نقطه از شبکه با کدام کلاس مطابقت دارد

```
from itertools import product
#Minima and maxima of both features
xmin, xmax = np.percentile(X[:, 0], [0, 100])
ymin, ymax = np.percentile(X[:, 1], [0, 100])
test_points = np.array([[xx, yy] for xx, yy in product(np.linspace(xmin,
xmax), np.linspace(ymin, ymax))])
#Predictions on the grid Azarnik
test_preds = grid_cv.predict(test_points)
X_1 = X[y == 1]
X_2 = X[y == 2]
%matplotlib inline
plt.figure(figsize=(10,7)) #change figure-size for easier viewing
plt.scatter(X_2[:,0],X_2[:,1], color = 'red', marker='^')
plt.scatter(X_1[:,0],X_1[:,1], color = 'blue')
colors = np.array(['b', 'r'])
plt.scatter(test_points[:, 0], test_points[:, 1],color=colors[test_preds-1],
alpha=0.25)
plt.title("RBF-separated classes")
```



شکل (۸-۳): ابر صفحه با هسته RBF

C و گاما

به طور بصری، پارامتر گاما تعیین می‌کند که یک نمونه در هر واحد فاصله چقدر تأثیرگذار باشد. اگر گاما کم باشد، در فواصل طولانی تأثیر دارند. اگر گاما زیاد باشد، تأثیر آن‌ها فقط در فواصل کوتاه است. SVM بردارهای پشتیبان را انتخاب می‌کند و گاما با شعاع

این بردارها رابطه معکوس دارد. مقدار کم C سطح تصمیمات را نرم‌تر می‌کند، در حالی‌که مقدار بزرگ C باعث می‌شود SVM همه مثال‌ها را به درستی طبقه‌بندی کند و منجر به سطوحی می‌شود که چندان صاف نیستند.

طبقه‌بندی چند کلاسه با SVM

ما شروع به گسترش دستور قبلی می‌کنیم تا انواع گل iris را بر اساس دو ویژگی طبقه‌بندی کنیم. این یک مسئله کلاسه‌بندی باینری نیست، بلکه یک مسئله کلاسه‌بندی چند کلاسه است. کلاسه‌بندی SVC در مورد چند کلاسه‌ها می‌تواند کمی تغییر کند. برای این منظور از هر سه کلاس مجموعه داده‌های iris استفاده خواهیم کرد.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
iris = datasets.load_iris()
X = iris.data[:, :2] #load the first two features of the iris data
y = iris.target #load the target of the iris data
X_0 = X[y == 0]
X_1 = X[y == 1]
X_2 = X[y == 2]
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2,random_state=7,stratify=y)
#OneVsRestClassifier
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.multiclass import OneVsRestClassifier
svm_est
=Pipeline([('scaler',StandardScaler()),('svc',OneVsRestClassifier(SVC()))])
Cs = [0.001, 0.01, 0.1, 1, 10]
gammas = [0.001, 0.01, 0.1, 1, 10]
```

شبکه پارامتر را بسازید. نام کلیدهای پارامتر در فرهنگ لغت در داخل خط لوله با svc__estimator__ شروع می‌شود.

```
param_grid = dict(svc__estimator__gamma=gammas,svc__estimator__C=Cs)
from sklearn.model_selection import RandomizedSearchCV
from sklearn.model_selection import StratifiedShuffleSplit
cv = StratifiedShuffleSplit(n_splits=5, test_size=0.2,random_state=7)
```

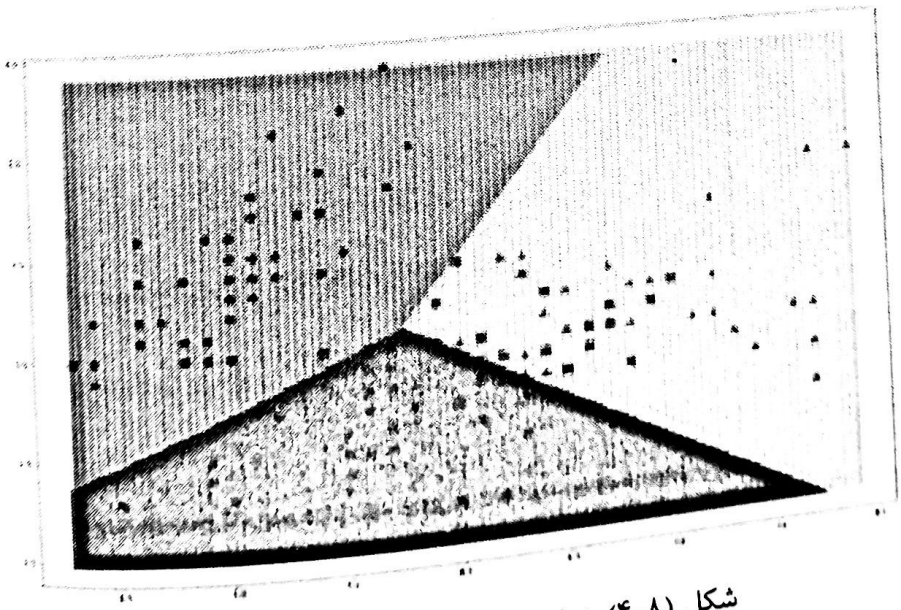
```
rand_grid = RandomizedSearchCV(svm_est,param_distributions=param_grid,
cv=cv,n_iter=10)
rand_grid.fit(X_train, y_train)
```

مقدار بهترین پارامترها عبارت است از

```
rand_grid.best_params_
output: {'svc__estimator__gamma': 0.01, 'svc__estimator__C': 10}
```

حال مرزها را نمایش می‌دهیم

```
%matplotlib inline
from itertools import product
#Minima and maxima of both features
xmin, xmax = np.percentile(X[:, 0], [0, 100])
ymin, ymax = np.percentile(X[:, 1], [0, 100])
#Grid/Cartesian product with itertools.product
test_points = np.array([[xx, yy] for xx, yy in product(np.linspace(xmin,
xmax,100), np.linspace(ymin, ymax,100))])
#Predictions on the grid
test_preds = rand_grid.predict(test_points)
plt.figure(figsize=(15,9)) #change figure-size for easier viewing
plt.scatter(X_0[:,0],X_0[:,1], color = 'green' , s=100)
plt.scatter(X_1[:,0],X_1[:,1], color = 'blue',marker='s' ,s=100)
plt.scatter(X_2[:,0],X_2[:,1], color = 'red' ,marker='^' ,s=100)
colors = np.array(['g', 'b', 'r'])
plt.tight_layout()
plt.scatter(test_points[:, 0], test_points[:, 1],
color=colors[test_preds],alpha=0.25)
```



شکل (۸-۴): نمایش مرزهای بین چند کلاس

کلاسه‌بند OneVsRestClassifier تعداد زیادی کلاسه بند باینری SVC را ایجاد می‌کند: برای هر کلاس در مقایسه با بقیه کلاس‌ها. در این حالت، سه مرز تصمیم‌گیری محاسبه می‌شود زیرا سه کلاس وجود دارد. درک این نوع طبقه‌بندی ساده است زیرا تعداد مرزها و سطح تصمیم‌گیری کم است. اگر ۱۰ کلاس وجود داشته باشد ۴۵= (۲ ÷ ۹)×۱۰ صفحه وجود دارد.

رگرسیون برداری پشتیبان(SVR)

روش‌های یادگیری با نظارت که داده‌ها را برای طبقه‌بندی و تجزیه و تحلیل رگرسیون تجزیه و تحلیل می‌کنند به عنوان رگرسیون پشتیبان شناخته می‌شوند. SVR بر اساس مفهوم Support Vector Machine یا SVM ساخته شده است. این یکی از مدل‌های معروف یادگیری ماشین است که می‌تواند در مسائل کلاسه‌بندی یا اختصاص کلاس‌ها هنگامی که داده‌ها به صورت خطی قابل تفکیک نیستند، مورد استفاده قرار گیرد.

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn import datasets
diabetes = datasets.load_diabetes()
X = diabetes.data
y = diabetes.target
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.2,random_state=7)
from sklearn.svm import SVR
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.multiclass import OneVsRestClassifier
svm_est
=Pipeline([('scaler',StandardScaler()),('svc',OneVsRestClassifier(SVR()))])
Cs = [0.001, 0.01, 0.1, 1]
gammas = [0.001, 0.01, 0.1]
param_grid = dict(svc__estimator__gamma=gammas,svc__estimator__C=Cs)
from sklearn.model_selection import RandomizedSearchCV
from sklearn.model_selection import StratifiedShuffleSplit
rand_grid = RandomizedSearchCV(svm_est,
param_distributions=param_grid,
cv=5,n_iter=5,scoring='neg_mean_absolute_error')
```

```
rand_grid.fit(X_train, y_train)
rand_grid.best_params_
{'svc__estimator__gamma': 0.1, 'svc__estimator__C': 1}
rand_grid.best_score_
0.9647765995975
```

امتیاز خیلی خوبی به نظر نمی‌رسد. الگوریتم‌های مختلف با تنظیمات مختلف را امتحان کنید و ببینید کدام یک بهترین عملکرد را دارد.